

Metal basics

<https://developer.apple.com/documentation/metal/performing-calculations-on-a-gpu>

1 Pipeline

1. Xcode Compiles to Intermediate Representation (IR), Not GPU Machine Code

When Xcode builds your app, it doesn't know what specific graphics chip it will run on. It might run on an Apple M1, M3 Max, an older A14 iPhone chip, or a future Apple Silicon processor. Each of these GPUs has a completely different machine architecture and instruction set.

- Xcode compiles your MSL (Metal Shading Language) text files into Metal IR (Intermediate Representation), which is a platform-independent bytecode.
- The final translation from Metal IR into the actual binary instructions that the local hardware can execute happens on the user's device at runtime.
- **CUDA Parallels:** This is just like CUDA compiling first to ptx – which is platform independent. Then, it compiles to platform specific binary at runtime (Just in time Compilation JIT).

2. The MTLFunction Object is just a Handler

When you load a function from that pre-compiled library in your app code, Metal creates an `MTLFunction` object. Because the code is still only in its intermediate (IR) state, this object cannot be directly executed. It serves as a proxy or lightweight reference that tells the Metal runtime: “Here is the name of the function, the entry point, its arguments, and the intermediate bytecode.”

3. Pipeline Creation performs the Final Compilation

To finally generate executable machine code, you pass that `MTLFunction` proxy into a pipeline descriptor and create a pipeline state object (like `MTLComputePipelineState`). During this step, the on-device driver takes:

- The Intermediate Representation (IR) embedded within your `MTLFunction`.
- The specific hardware architecture of the host device's GPU.
- State information (e.g., pixel formats, vertex layouts, or threadgroup configurations).

The driver then bakes all of this together, performing the final hardware-specific compilation. The resulting pipeline state object holds the actual executable code, which is why creating a pipeline is computationally expensive and why pre-compiled `.metallib` files alone aren't enough to run immediately.

2 Command Buffer and Computer Encoder

2.1 Create a Command Buffer

- A command buffer (`MTLCommandBuffer`) is a container that stores a list of compiled GPU commands.
- You fetch a command buffer from a command queue (`_mCommandQueue`). Think of the command queue as a pipeline to the GPU, and the command buffer as an empty train car waiting to be loaded with cargo (commands).
- Commands are not executed immediately when written. Instead, they are queued up inside this buffer so they can all be submitted to the GPU as a single, efficient batch.

2.2 Create a Command Encoder

- A command encoder is a specialized tool used to write commands into the command buffer. You cannot write commands directly to a command buffer; you must use an encoder.
- Depending on the type of task, you use a specific type of encoder:
 - **Compute Command Encoder** (`MTLComputeCommandEncoder`): Used for data processing, math, or machine learning (as described in the text).
 - **Render Command Encoder**: Used for drawing graphics to the screen.
 - **Blit Command Encoder**: Used for copying data between memory resources.
- In short, the job of a Command Encoder (`MTLCommandEncoder`) is to translate high-level API commands written in Objective-C, Swift, or C++ into low-level, hardware-specific binary commands that the GPU can understand.

2.3 The Compute Pass

When you use a compute command encoder, you are creating a compute pass. This pass defines:

- What functions (shaders) to run.
- What data (buffers) those functions should read from and write to.
- The layout of the threads (the execution grid) on the GPU.